Assignment IV



Luke Profio

Why Predict ICU LOS?

- Helps with the allocation of beds, staff scheduling and OR planning
- Improves the control of costs and flow of patients throughout the hospital
- MIMIC-III has over 50,000 ICU stays!
- In this tutorial, we treat LOS prediction as a **regression** problem (days)

1. Prerequisites

- Python >= 3.8, pandas, scikit-learn & numpy
- Local .CSV copies of PATIENTS, ADMISSIONS & ICUSTAYS (MIMIC-III v1.4)
- *Pip install pandas scikit-learn*, as needed
- Set your working directory to the folder containing the .CSV files

2. Load & Inspect Relevant Tables

```
patients   = pd.read_csv('PATIENTS.csv',   parse_dates=['DOB'])
admissions = pd.read_csv('ADMISSIONS.csv', parse_dates=['ADMITTIME','DISCHTIME','DEATHTIME'])
icustays   = pd.read_csv('ICUSTAYS.csv',   parse_dates=['INTIME','OUTTIME'])
print(patients.shape, admissions.shape, icustays.shape)
```

- Here we parse all the columns up-front.
- There should be over 40,000 patients and 50,000 ICU stays.
- This helps to catch corrupted files or wrong extracts early on before downstream code fails.

3. Merge Tables into One Row per ICU Stay

```
df = icustays.merge(admissions, on=['SUBJECT_ID','HADM_ID']) \
        .merge(patients[['SUBJECT_ID','DOB','GENDER']], on='SUBJECT_ID')
print(df.columns[:12])
print('Rows after merge:', df.shape[0])
```

- Inner joins guarantee that every row has patient, admission, and ICU information.
- The output should return over 50,000 rows.
- By consolidating these tables into one that is coherent, we simplify the later preprocessing of data and prevention of mismatched records.

4. Feature Engineering

```
df['AGE'] = ((df['INTIME']-
df['DOB']).dt.days/365.2425).clip(upper=90)

df = df[df['AGE']>=16]            # adult cohort

df['ETHNICITY_GROUPED'] =
df['ETHNICITY'].apply(group_ethnicity)


cat_cols = ['GENDER','ADMISSION_TYPE','INSURANCE',
        'ETHNICITY_GROUPED','FIRST_CAREUNIT']

df[cat_cols] = df[cat_cols].astype('category')
```

- We cap age at 90.
- Ethnicity is collapsed into 5 labels via *group_ethnicity()*.
- We also drop neonatals and obvious timestamp errors.
- This transforms the raw data to meaningful predictors the model can learn from.

5. Target Variable Calculation

```
df['ICU_LOS_DAYS'] = (df['OUTTIME']-df['INTIME']).dt.total_seconds()/86400
df = df[(df['ICU_LOS_DAYS']>=0) & df['DEATHTIME'].isna()]
```

- Here we calculate the target variable, LOS in days.
- All of the negative rows are removed.
- We exclude in-ICU deaths to more effectively model discharges.
- Accurate ground-truth labels are very important for supervised learning; any error here can adversely affect the model.

6. Data Pre-Processing

```
df = df.dropna(subset=['AGE','ICU_LOS_DAYS'])
X_full = pd.get_dummies(df.drop('ICU_LOS_DAYS',axis=1), drop_first=False)
y_full = df['ICU_LOS_DAYS']

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(
    X_full,y_full,test_size=0.2,random_state=42)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train['AGE'] = scaler.fit_transform(X_train[['AGE']])
X_test['AGE']  = scaler.transform(X_test[['AGE']])
```

6. Data Pre-Processing

- Here we pre-process the data by one-hot encoding all categories into one line.
- Additionally, we standardize **Age** using only the training statistics.
- Here we create a numeric matrix that is compatible with scikit-learn’s underlying technology.

7. Training the Random Forest Model

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=100,
                           random_state=42,
                           n_jobs=-1)
rf.fit(X_train, y_train)
```

- Here, we compose over 100 trees using all CPU cores (*n_jobs = -1*)
- Training should take < 1 minute on a laptop; may scale to the full dataset without change.
- Random Forest builds multiple decision trees on randomly sampled data, averaging their predictions and reducing over-fitting. It is ideal for capturing non-linear patterns most data have.

8. Evaluate on Hold-Out Test Set

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_pred = rf.predict(X_test)
print('MAE:', mean_absolute_error(y_test,y_pred))
print('RMSE:', mean_squared_error(y_test,y_pred, squared=False))
print('R^2 :', r2_score(y_test,y_pred))
```

- Here, we interpret MAE/RMSE in **days** and evaluate on the hold-out testing set.
- This assesses the model's performance outside of the training data.
- We compare to the baseline and predict median LOS.

9. Feature Importance

```
imp = pd.Series(rf.feature_importances_, index=X_train.columns).sort_values(ascending=False)
print(imp.head(10))
```

- A key component is gauging the importance of each feature, in other words how strong a predictor the feature is.
- You may find certain variables such as Age, First Care Unit and Admission Type to be among the highest predictors.
- Here we use importances for clinical interpretation or further design of features.
- This helps gain clinician trust in the model.

Optional Extensions

- Additionally, you can incorporate vitals, labs, and ICD codes for an enhanced signal.
- You can also try Gradient Boosting (e.g. XGBoost, LightGBM) which may offer higher accuracy compared to the models in this tutorial.
- Prolonged stay risk or prediction intervals can also be incorporated.
- The entire pipeline could also be packaged via *sklearn.pipeline* for deployment.

Q&A

Thank You!